INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABORATORY

PROJECT TITLE: PUBLIC-TRANSPORT PUNCTUALITY MONITOR

TEAM MEMBERS:

No.	NAME	ROLL NO.	ENROLLMENT NO.
1.	Srinjay Mukherjee	255	12025052003274
2.	Prithwish Chakraborty	254	12025052003273
3.	Shankhanil Ghosh	256	12025052003275
4.	Amit Majumder	287	12025052003308
5.	Asmijit Das	265	12025052003285

SECTION: D1

SUBMITTED TO: Prof. Indranil Basu

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINEERING

ACADEMIC SESSION: 2026-2027

DATE OF SUBMISSION: _____

PUBLIC-TRANSPORT PUNCTUALITY MONITOR

Abstract

This project develops a Python program that monitors the punctuality of a small shuttle service by comparing scheduled and actual arrival times at each stop. It validates time inputs in HH:MM form, computes the delay of every stop arrival, classifies arrivals as early, on time, late or severely late, stores route and stop records in CSV files, and exports summaries as JSON. The implementation combines object-oriented programming with a Trip class, standard CSV and JSON handling, NumPy for delay distributions, Pandas for route-wise and day-wise summaries, and Matplotlib for delay bar charts and time-series plots. A Tkinter desktop dashboard presents the results interactively with route and period filters.

The included demonstration dataset is synthetically generated, not collected from a real transport operator. From 1,349 input rows, 1,344 pass validation and five are rejected. Across the valid records the mean delay is 3.66 minutes, the median is 3.00 minutes, and 73.4% of arrivals are on time (within 1 minute early to 5 minutes late). Route R3 is the least punctual (51.3% on time) and route R1 the most punctual (94.6% on time). Peak-hour arrivals are later on average than off-peak arrivals (4.35 versus 2.60 minutes), although a few severe incident trips distort the simple averages. The results describe the behaviour of the software on simulated data and are not evidence about any real service.

Keywords

Public transport; punctuality; schedule adherence; delay analysis; Python; Trip class; CSV; JSON; NumPy; Pandas; Matplotlib; Tkinter; time series.

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Software and System Requirements
4. System Design and Methodology
5. Implementation of Major Modules
6. Results and Discussion
7. Testing and Validation
8. Limitations and Future Scope
9. Conclusion
10. References
11. Appendix: CSV Format and Execution

1. Introduction

Passengers judge a public-transport service largely by whether vehicles arrive when the timetable says they will. Operators usually hold scheduled and observed arrival times in spreadsheets, where comparing routes, days and times of day by hand is slow and error-prone. A small analytical program can validate the raw times, compute delays consistently and present summaries that make recurring problems visible.

The developed program reads scheduled and actual arrival times for shuttle trips, computes the delay at every stop, and reports how punctual each route and each day was. It can run on the built-in synthetic dataset or on a user-supplied CSV file with the required columns. Results are written to JSON and CSV files and to four chart images.

2. Problem Statement and Objectives

Problem statement: Analyse scheduled and actual arrival times from a small bus or shuttle dataset. Compute delays and validate time inputs; use NumPy for delay distributions; maintain stop and route records in files; implement a Trip class; use Pandas for route and day summaries and JSON export; and draw delay bars and time-series plots.

The principal objectives are:

- Validate time inputs (HH:MM, 00:00 to 23:59) and reject malformed or inconsistent records with a stated reason.
- Implement a Trip class that encapsulates a stop arrival, its validation, its delay and its punctuality status.
- Maintain route and stop records in dictionaries and CSV files, and read and write trip records in CSV form.
- Use NumPy to describe delay distributions: mean, median, standard deviation, percentiles, histogram bins and interquartile-range outliers.
- Use Pandas to summarise punctuality by route, by day, by stop and by peak or off-peak period, and export the summaries as JSON.
- Draw delay bar charts and time-series plots, and interpret the findings cautiously.

3. Software and System Requirements

Component	Purpose
Python 3	Core program execution
csv, json, random, datetime	File handling, JSON export, reproducible sample generation and date checks
NumPy	Arrays, percentiles, histogram bins and outlier fences
Pandas	Tabular records, group-by summaries, pivot tables and rolling means
Matplotlib	Delay bar charts, histograms and time-series plots, shown in the app or saved as PNG files
Tkinter / ttk	Desktop graphical user interface (punctuality_app.py)
Python IDLE	Run the program or the app using Run Module (F5)

The program is suitable for any desktop system with a working Python 3 installation and the three third-party libraries. The analysis module needs no graphical window and writes its charts to image files; the optional desktop app uses Tkinter, which is normally supplied with standard Python installations, subject to platform packaging.

4. System Design and Methodology

4.1 Input data and schema

Each trip record describes one stop arrival and contains six fields: trip_id (identifier of one vehicle run), route_id (R1, R2 or R3), service_date (YYYY-MM-DD), stop_name, scheduled (planned arrival, HH:MM) and actual (observed arrival, HH:MM). Delay and status are derived by the program rather than supplied in the file. Route and stop records are kept separately in routes.csv with the columns route_id, route_name, stop_order, stop_name and offset_min (minutes after the route start).

4.2 Data preparation

On start-up, if no CSV is supplied, the program creates a reproducible synthetic dataset with a fixed random seed (2026): three campus shuttle routes, four stops each, 14 days from 7 to 20 September 2026, ten weekday departures and three weekend departures per route per day. This gives 1,344 valid stop arrivals. Five deliberately faulty rows are appended to exercise validation. Each row is passed to the Trip class; rows with an impossible time, a blank time, an unknown route, a stop that does not belong to the route, or a badly formatted date are skipped and reported with the CSV row number.

4.3 Analysis workflow

1. Read the route records and the trip records (or generate the sample data).
2. Validate each row using the Trip class and keep the valid trips.
3. Compute each delay in minutes and assign a punctuality status.
4. Build a Pandas DataFrame and add route name, hour, weekday and peak or off-peak period.
5. Compute NumPy statistics for the delay array, including histogram bands and IQR fences.
6. Aggregate by route, day, stop and period with Pandas group-by and pivot tables.
7. Export the summaries to summary.json and the detailed table to CSV, and draw the charts.

4.4 Statistical measures

The delay of a stop arrival is the actual time minus the scheduled time, in minutes; positive values mean late. Differences beyond twelve hours are wrapped by 24 hours so that a trip crossing midnight is handled correctly. The mean and median describe the centre of the delay distribution, the sample standard deviation its spread, and the 90th percentile (P90) the delay exceeded by one arrival in ten. The on-time percentage is the share of arrivals with a delay from −1 to +5 minutes inclusive.

Outliers are flagged with the interquartile-range rule: with IQR = Q3 − Q1, a delay is flagged when it falls below Q1 − 1.5(IQR) or above Q3 + 1.5(IQR). Flagged values are statistical extremes, not necessarily errors. A 3-day rolling mean of the daily average delay is used to smooth the time series.

5. Implementation of Major Modules

Module / function	Functionality
parse_time(), format_time()	HH:MM validation and conversion to minutes after midnight
compute_delay(), classify()	Delay calculation (with midnight wrap) and status labelling
Trip, from_dict(), to_dict()	Object-oriented trip record, field validation and conversion
default_routes(), save_routes(), load_routes()	Route and stop records held in a dictionary and stored in routes.csv
generate_sample_data(), validate_rows()	Synthetic data, input validation and rejection reasons
load_trips_csv(), save_trips_csv()	Reading (DictReader) and writing trip records
delay_stats(), iqr_bounds(), delay_histogram()	NumPy statistics, outlier fences and histogram bands
build_dataframe(), route_summary(), day_summary(), period_summary(), stop_summary()	Pandas tables and group-by aggregation
export_json()	JSON export of overall, route, day and period summaries
fig_route_bars(), fig_distribution(), fig_time_series(), fig_peak_hours()	Matplotlib delay bars, distribution and time-series figures (shown in the app or saved as PNG)
PunctualityApp, _build(), refresh(), load_rows()	Tkinter dashboard: layout, filters, metric cards, embedded charts and records table
open_csv(), use_sample(), export_json(), save_charts()	App actions: load a user file, reload sample data, export the summary and chart images
run()	Runs the complete pipeline and prints the report to the console

5.1 Program layout, interface and outputs

The program is a single script, punctuality_monitor.py, run from IDLE or the command line, with an optional path to a trip CSV file. It prints the validation result, the overall statistics, the route, period and day summaries, and the outlier table. It then writes routes.csv, trips.csv, trips_with_delays.csv, summary.json and four PNG figures to an output folder. A separate unit-test file, test_punctuality.py, checks the main functions.

The desktop app, punctuality_app.py, offers the same analysis through a window. The top bar has Open CSV and Use sample data buttons, a route selector, a period selector (all, peak or off-peak), Save charts and Export JSON. Five metric cards show the number of stop arrivals, mean delay, median delay, on-time percentage and number of severely late arrivals. Six tabs show the route comparison, delay distribution, time series, delay by hour, a records table with outliers highlighted, and a written summary. Rows rejected during a CSV import are listed in a message box.

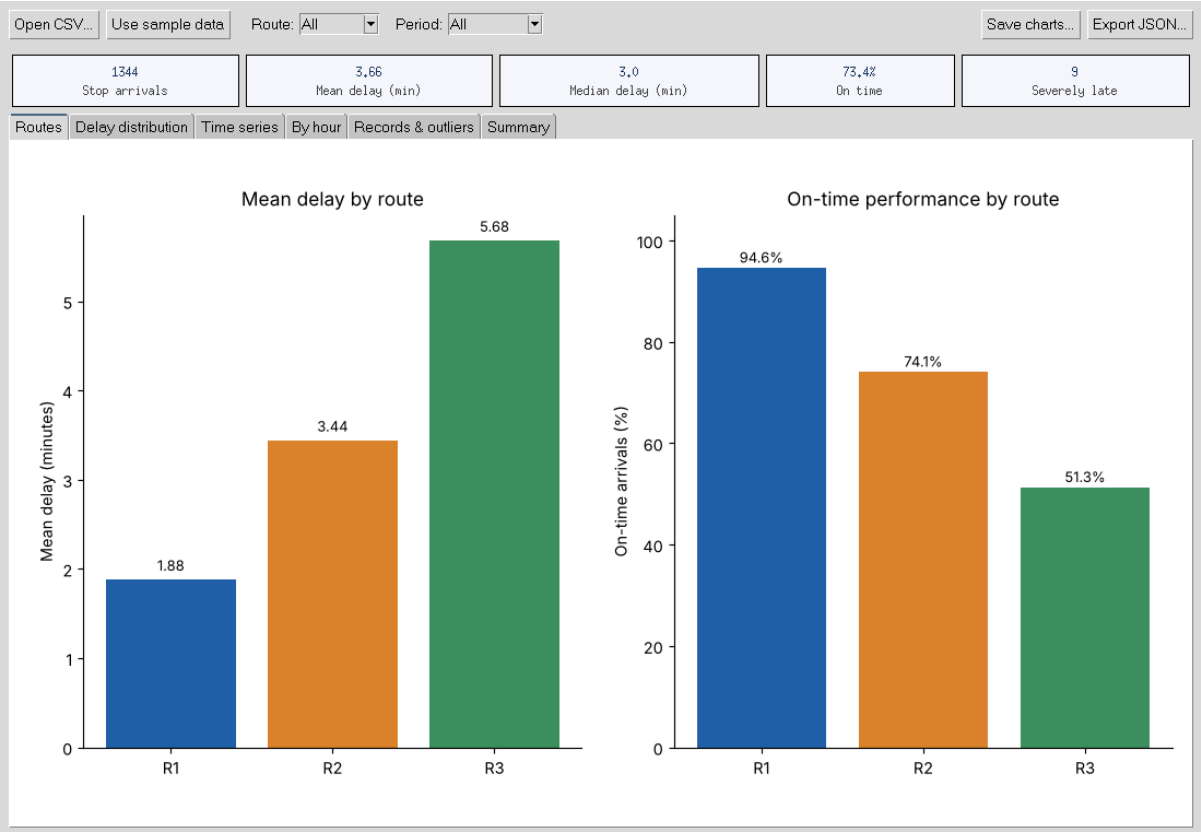


Figure 1. The desktop app showing the route comparison for the sample data.

5.2 Punctuality classification

Status thresholds: Early = more than 1 minute before schedule (delay below −1); On time = delay from −1 to +5 minutes; Late = delay of 6 to 15 minutes; Severely late = delay above 15 minutes. Whole minutes are used because input times are recorded to the minute. The thresholds are adjustable constants at the top of the program and may differ from an operator's own standard.

6. Results and Discussion

6.1 Demonstration dataset and descriptive findings

The following results were computed from the fixed-seed synthetic dataset generated by the submitted program. They will differ if a separate CSV is imported or the thresholds are changed.

Indicator	Observed value
Input rows	1,349
Valid stop arrivals	1,344
Rejected rows	5
Mean delay	3.66 minutes
Median delay	3.00 minutes
Sample standard deviation of delay	4.52 minutes
90th percentile (P90) of delay	8.00 minutes
Maximum delay / minimum delay	51 minutes / −3 minutes
On-time arrivals (−1 to +5 min)	73.4%
Severely late arrivals (> 15 min)	9 of 1,344 (0.7%)
Q1 / Q3 of delay, IQR	1.0 / 6.0 minutes, IQR = 5.0
IQR fences (k = 1.5)	−6.5 to 13.5 minutes
Outliers flagged	13 of 1,344 (1.0%)

By status, 986 arrivals are on time, 332 late, 17 early and 9 severely late. Most delays fall between 0 and 10 minutes, with a small tail caused by three incident trips.

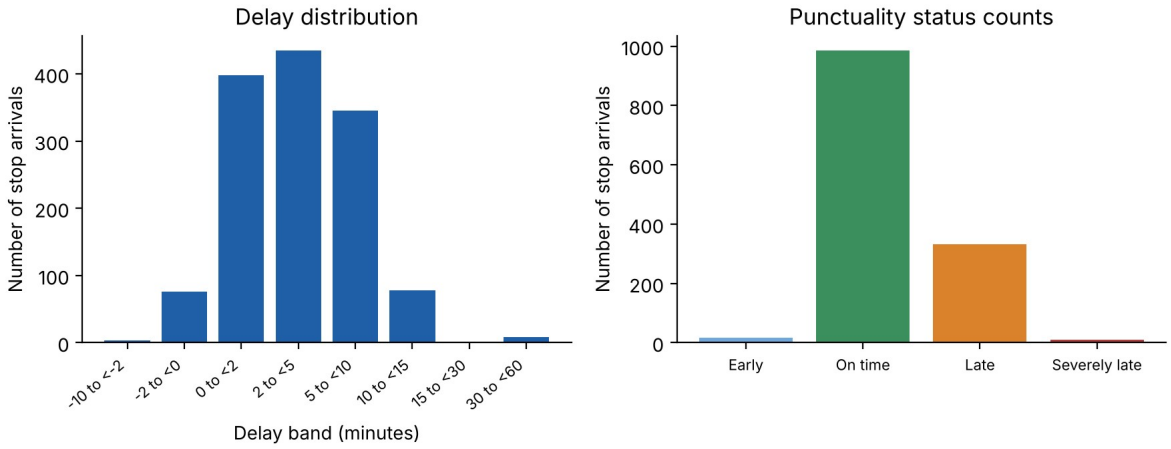


Figure 2. Distribution of delays and punctuality status counts for the generated sample.

6.2 Route-wise summaries

Route	Name	Records	Mean delay	P90	On time %	Severe %
R1	Campus – Sector V	448	1.88	5.0	94.6	0.00
R2	Campus – Esplanade	448	3.44	8.0	74.1	0.67
R3	Campus – Howrah	448	5.68	11.0	51.3	1.34

In this demonstration sample, R1 has the lowest mean delay (1.88 minutes) and the highest on-time share (94.6%), while R3 has the highest mean delay (5.68 minutes) and only 51.3% of arrivals on time. Route R3 is also the longest, so delay has more distance in which to accumulate. Within each route the average delay grows from the first stop to the last (for R3, 0.38 minutes at Campus Gate to 7.65 minutes at Howrah Station), so lateness is passed down the route rather than created again at every stop. These are properties of the simulated data and should not be read as a ranking of real routes.

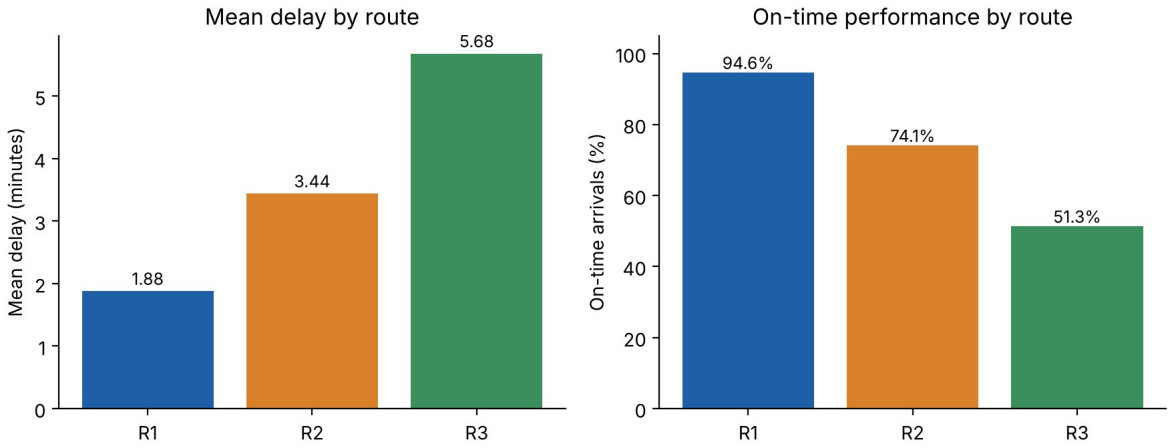


Figure 3. Mean delay and on-time percentage for each route.

6.3 Time-of-day and day-wise patterns

Period	Records	Mean delay	Median	On time %	Severe %
Peak (07–09 h, 17–19 h)	816	4.35	4.0	63.5	0.37
Off-peak	528	2.60	2.0	88.6	1.14

Peak-period arrivals are later on average (4.35 versus 2.60 minutes) and are on time far less often (63.5% versus 88.6%). Interestingly, the off-peak group has a higher severe-lateness share, because two of the three simulated incident trips (at 12:00 and 14:00) happened to be scheduled off-peak. When these three incident trips are set aside, the off-peak mean delay falls to 2.18 minutes and the peak mean to 4.22 minutes, so the peak-hour effect is not produced by the incidents. Weekday arrivals average 3.87 minutes late and weekend arrivals 1.98 minutes, with fewer trips running at weekends.

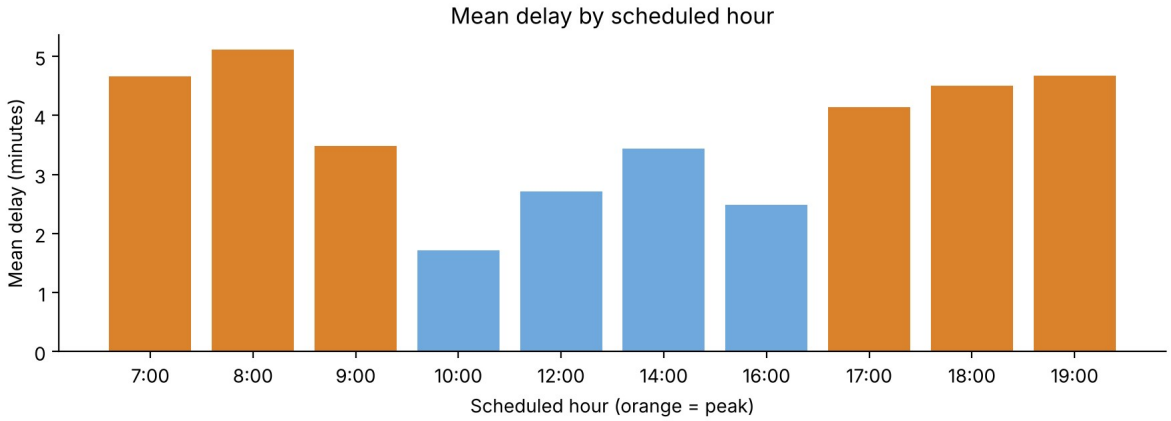


Figure 4. Mean delay by scheduled hour of day; peak hours are shown in orange.

The time-series plot shows the daily mean delay of each route. R3 is the most variable (daily means between about 2.4 and 9.0 minutes), with spikes on 8 and 17 September that correspond to incident trips. Daily means for all routes together stay between about 1.3 and 4.6 minutes, and the 3-day rolling mean is steady at roughly 2.5 to 4.2 minutes, with lower values on the weekend of 12–13 September.

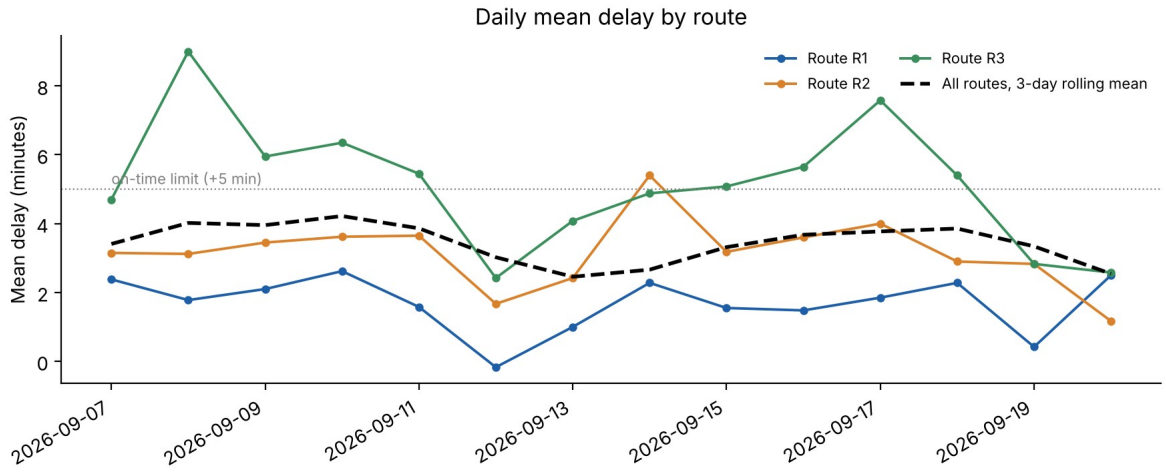


Figure 5. Daily mean delay by route with a 3-day rolling mean.

6.4 Outliers and interpretation

Trip ID	Route	Date	Stop	Scheduled	Actual	Delay (min)
T0056	R3	2026-09-08	Park Circus	14:15	15:06	51
T0056	R3	2026-09-08	Park Street	14:28	15:18	50
T0056	R3	2026-09-08	Howrah Station	14:48	15:37	49
T0183	R2	2026-09-14	Ultadanga	12:14	12:45	31
T0183	R2	2026-09-14	Sealdah	12:26	12:55	29
T0183	R2	2026-09-14	Esplanade	12:40	13:10	30
T0280	R3	2026-09-17	Park Circus	08:15	08:57	42
T0280	R3	2026-09-17	Park Street	08:28	09:09	41
T0280	R3	2026-09-17	Howrah Station	08:48	09:31	43

Table 6.4 lists the severely late arrivals. Four further records are flagged by the IQR rule (delays of 14 minutes on trips T0089 and T0119, all on route R3 in the evening peak), making 13 flagged records in total.

The IQR rule identifies statistical extremes, not necessarily mistakes. Each of the three severe trips has a delay that carries through to every later stop of the same trip, which is what would be expected of a single breakdown or road closure. In a real dataset such records should be investigated with the operator before being removed, and removing them is a sensitivity check rather than a cleaning requirement.

The analysis is descriptive. It shows that, in the sample, late arrivals are more common at peak hours and on route R3, but it does not show why. Traffic volume, route length, passenger loading, driver changes, road works and weather could each contribute. Moreover, the built-in records are generated by a mathematical procedure, so the numerical results are examples of program behaviour rather than empirical evidence about any real service.

7. Testing and Validation

Test case	Expected behaviour	Observed
Valid time 08:05 / 23:59	Convert to 485 / 1439 minutes	Passed
Invalid times 25:61, blank, 9.30, 24:00	Raise ValueError	Rejected
Delay 10:00 → 10:07; 10:00 → 09:58	+7 and −2 minutes	Passed
Midnight crossing 23:55 → 00:10	Delay of +15 minutes	Passed
Status boundaries (−3, 0, 5, 6, 16)	Early, On time, On time, Late, Severely late	Passed
Trip with bad date or empty ID	Reject record	Rejected
Sample start-up	Build records and summaries	1,344 accepted; 5 rejected
Fixed random seed	Identical data on every run	Passed
NumPy statistics on [0, 2, 4, 6, 20]	Mean 6.4, median 4, on-time 60%	Passed
Histogram band counts	Sum equals number of records	1,344 = 1,344
Route and day summaries	3 routes, 14 days; counts add up	Passed
CSV and routes round trip	Records unchanged after save and load	Passed
CSV with missing column	Report missing columns	Rejected
JSON export	File contains overall, route, day and rejected counts	Passed
App start-up (sample data)	Five metric cards and six tabs populated	1,344 arrivals; 73.4% on time
App filter: route R3, Peak	Recalculate cards and charts	272 arrivals; mean 6.70 min; 34.6% on time
App CSV load	Report valid and rejected rows	1,344 valid; 5 rejected

The 14 unit tests in test_punctuality.py were run with Python's unittest module and all passed. Charts and the console report were checked by inspecting the generated output files, and the desktop app was run on a virtual display to confirm the cards, tabs, filters and CSV loading. The app should also be run once on the computer used for the presentation.

8. Limitations and Future Scope

Limitations: (i) the included observations are synthetic, (ii) times are recorded only to the minute, (iii) the analysis is descriptive rather than predictive, (iv) the on-time thresholds and the peak-hour definition are fixed and may not match an operator's standard, (v) the sample covers only 14 days and three routes, and (vi) the program does not consider passenger demand, traffic or weather.

Possible extensions include add and edit forms in the desktop app, live data from a vehicle-tracking feed, seconds-level timestamps, headway-based measures for high-frequency routes, an integrated local database, automatic detection of recurring bottleneck stops, comparison of weekdays and holidays over longer periods, and prediction of delay from time of day and route. Any real operator data should be used with permission and in line with the operator's data policy.

9. Conclusion

The project demonstrates a complete workflow for monitoring shuttle punctuality: validated time inputs, an object-oriented Trip record, route and stop records in files, NumPy delay statistics, Pandas route and day

summaries, JSON export, and Matplotlib bar and time-series charts. On the included synthetic data, 73.4% of arrivals are on time, route R3 and the peak hours are the weakest, and three incident trips account for most of the severe delays. The conclusion is cautious: the program can reveal and communicate where and when delays occur, but it cannot by itself explain their causes.

10. References

Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

NumPy Developers. NumPy Documentation. https://numpy.org/doc/

Pandas Development Team. pandas Documentation. https://pandas.pydata.org/docs/

Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/

Python Software Foundation. csv and json modules. https://docs.python.org/3/library/csv.html and https://docs.python.org/3/library/json.html

11. Appendix: CSV Format and Execution

Required input CSV headers (exact spelling): trip_id, route_id, service_date, stop_name, scheduled, actual. Times use the 24-hour HH:MM form and dates use YYYY-MM-DD. Route and stop names must exist in routes.csv.

trip_id	route_id	service_date	stop_name	scheduled	actual
T0001	R1	2026-09-07	Campus Gate	07:00	07:01
T0001	R1	2026-09-07	Salt Lake Stadium	07:12	07:15
T0001	R1	2026-09-07	Sector V	07:35	07:38

To open the desktop app, open punctuality_app.py in IDLE and select Run → Run Module (F5); use Open CSV to import a file. To run the analysis without a window, open punctuality_monitor.py and run it the same way. To analyse a file of your own, run python punctuality_monitor.py trips.csv from a command prompt. Install the required third-party packages with: py -m pip install numpy pandas matplotlib. The analysis script writes to an output folder containing routes.csv, trips.csv, trips_with_delays.csv, summary.json and the four figure files. To run the tests, use: python -m unittest test_punctuality.